Année scolaire 2026-2027

Lycée Victor Hugo - Colomiers

Première NSI

AP

Algorithmes et Programmation
Synthèse

Thomas Ricart

ricart.lvh@laposte.net

https://thomasricart.github.io

6 avril 2026

AP01-S CONSTRUCTIONS ÉLÉMENTAIRES

Définition 1 : Les différents types de variables

- **int** : (entiers naturels) Ex. : `a = 5`
- **float** : (nombres décimaux) Ex. : `b = 3.14`
- **str** : (chaînes de caractères) Ex. : `c = "Bonjour"`
- **bool** : (valeurs booléennes) Ex. : `d = True`
- **list** : (listes) Ex. : `e = [1, 2, 3]`
- **tuple** : (tuples) Ex. : `f = (1, 2, 3)`

Opérations mathématiques de base en Python

`+` addition : `5 + 3` donne 8

`-` soustraction `5 - 3` donne 2

`*` multiplication `5 * 3` donne 15

`/` division `6 / 2` donne 3.0 (`float`)

`**` puissance `2 ** 3` donne 8

`%` reste de la division entière `7 % 3` donne 1

`//` quotient de la division entière `7 // 3` donne 2

Propriété 1 : Les chaines de caractères

Indice	0	1	2	3	4	5	6
Caractère	B	o	n	j	o	u	r
Indice Négatif	-7	-6	-5	-4	-3	-2	-1

Code Python
```python
mot = 'Bonjour'
mot[0]    # Affiche 'B'
mot[4]    # Affiche 'o'
len(mot)  # Affiche 7
```

Les opérations sur les chaines de caractères

`+` pour la concaténation : `'Bonjour' + ' a tous'` donne `'Bonjour a tous'`

`*` pour la répétition : `'Bonjour' * 3` donne `'BonjourBonjourBonjour'`

`[]` pour l'indexation : `'Bonjour'[0]` donne `'B'`

`in` pour le test d'appartenance : `'on' in 'Bonjour'` donne `True`

`not in` pour le test d'appartenance : `'on' not in 'Bonjour'` donne `False`

`len()` pour la longueur : `len('Bonjour')` donne 7

Définition 2 : Les booléens

Code Python
```python
a = True
b = False
c = 6
d = 3
```

Code Python
```python
a == b  # Affiche False
a != b  # Affiche True
c == d  # Affiche False
```

Code Python
```python
c <= d   # Affiche False
c > d    # Affiche True
c >= d // 2  # Affiche True
```

Table de vérité du `not`

Expression	Résultat
`not` True	False
`not` False	True

Table de vérité du `and`

Expression	Résultat
True `and` True	True
True `and` False	False
False `and` True	False
False `and` False	False

Table de vérité du `or`

Expression	Résultat
True `or` True	True
True `or` False	True
False `or` True	True
False `or` False	False

AP02-S LES FONCTIONS

Gestion des entrées en Python

Code Python

```python
a = input("Entrer une valeur au clavier")          # a sera de type str
b = int(input("Entrer une valeur au clavier"))     # b sera de type int
```

Code Python

```python
a = 6
print(a)  # Affiche 6
print("La valeur de a est", a)  # Affiche "La valeur de a est 6"
```

Méthode 1 : Formatage de l'affichage avec `f-string` et `format()`

L'idée est de créer un texte à trou que l'on va compléter par des variables.

```
>>> name = 'Paul'
>>> age = 25
>>> print(f'Votre nom est {name} et vous avez {age} ans.')
Votre nom est Paul et vous avez 25 ans.
```

```
>>> print('Votre nom est {} et vous avez {} ans'.format(name, age))
Votre nom est Paul et vous avez 25 ans
```

Définition 1 : Les fonctions

Code Python

```python
def f(x):
    """ Fonction renvoyant x ** 2 """
    y = x ** 2          # Calcul de x ** 2
    return y            # valeur renvoyée: y
```

Code Python

```python
a = 4            # on affecte 4 à la variable a
z = f(4)         # on appelle la fonction f avec comme argument 4
                 # et on récupère la valeur renvoyée dans la variable z
```

Remarque : Variables locales et variables globales

- les **variables locales** ne sont connues que dans la fonction
- les **variables globales** sont connues dans tout le programme

Code Python

```python
print(x)    # ERREUR -> x est une variable locale (tout comme y et z)
```

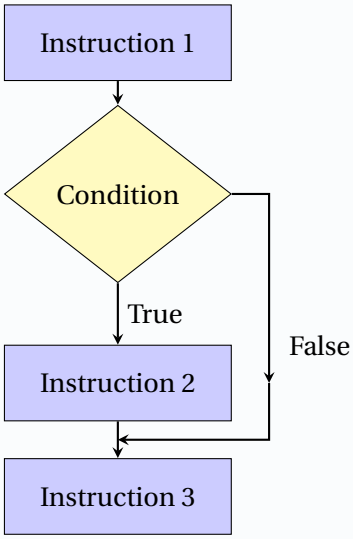
Vérification de la valeur d'une variable

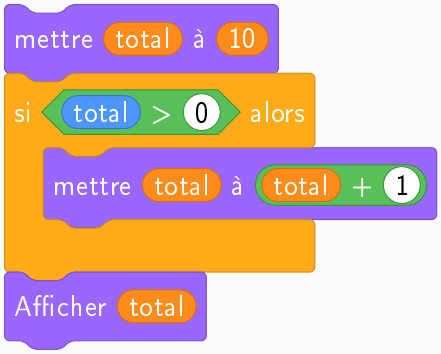
Code Python

```python
assert f(6) == 36          # True -> aucune message
assert f(6) == 12          # False -> message d'erreur
assert not(f(6) == 12)     # True -> aucun message
```

AP03-S CONDITIONS IF

Les structures conditionnelles IF





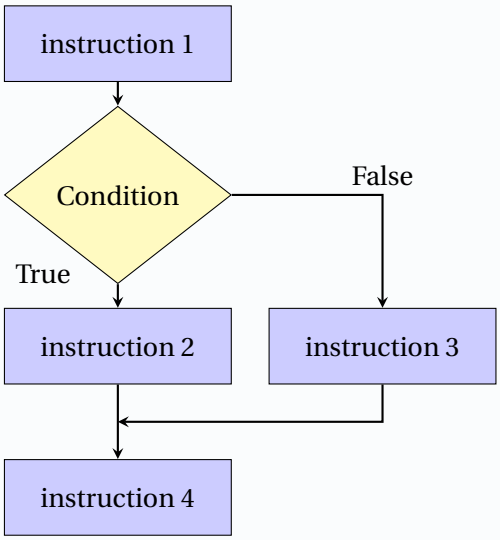

```python
# En python
Instruction_1
if condition:
    Instruction_2
Instruction_3
```

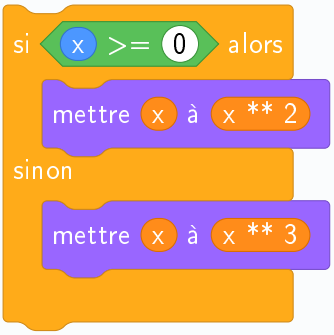
```python
# En python
total = 10
if total > 0:
    total = total + 1
print(total)
```

- l'`Instruction 1` est effectuée
- SI la `Condition` est `True` alors l'`Instruction 2` est effectuée
- SINON on va directement à l'`Instruction 3`
- Dans tous les cas l'`Instruction 3` est effectuée

La `Condition` est un test qui est évalué en python par un booléen.

Les structures conditionnelles IF – ELSE






```python
# En python

instruction_1

if condition:
    instruction_2
else:
    instruction_3
instruction_4
```

- l'`instruction 1` est effectuée
- SI la `Condition` est `True` alors l'`instruction 2` est effectuée
- SINON l'`instruction 3` est effectuée
- Dans tous les cas l'`instruction 4` est effectuée

Exemple : Les structures conditionnelles

Code Python

```python
def plus_grand2(a, b):
    ''' Renvoie le plus grand des
    deux nombres a et b '''
    if a > b:
        return a
    else:
        return b

maximum = plus_grand2(5, 10)
print(maximum) # Affiche 10
```

Code Python

```python
def plus_grand3(a, b, c):
    ''' Renvoie le plus grand des
    trois nombres a, b et c '''
    if a > b and a > c:
        return a
    elif b > a and b > c:
        return b
    else:
        return c

maximum = plus_grand3(5, 10, 7)
print(maximum) # Affiche 10
```

AP04-S BOUCLES FOR ET WHILE

La boucle FOR en python

Code Python
```python
for i in range(0, 7):
    print(i, end=" ")
# affichage:
# 0 1 2 3 4 5 6
```

Code Python
```python
for i in range(2, 7):
    print(i, end=" ")
# affichage:
# 2 3 4 5 6
```

Code Python
```python
for i in range(5):
    print(i, end=" ")
# affichage:
# 0 1 2 3 4
```

Itération sur une chaine de caractères

Code Python
```python
mot = 'Hello'
for i in range(0, len(mot)):
    print(mot[i])
```

Code Python
```python
mot = 'Hello'
for lettre in mot:
    print(lettre)
```

Algorithmes de Recherche dans un itérable

Code Python
```python
def est_present(mot, lettre_cherchee):
    for lettre in mot:
        if lettre == lettre_cherchee:
            return True
    return False
```

Code Python
```python
def est_present(mot, lettre_cherchee):
    for i in range(len(mot)):
        if mot[i] == lettre_cherchee:
            return True
    return False
```

Algorithmes de comptage dans un itérable

Code Python
```python
def compte(mot, lettre_cherchee):
    c = 0
    for lettre in mot:
        if lettre == lettre_cherchee:
            c = c + 1
    return c
```

Code Python
```python
def compte(mot, lettre_cherchee):
    c = 0
    for i in range(len(mot)):
        if mot[i] == lettre_cherchee:
            c = c + 1
    return c
```

Algorithmes de somme

Code Python
```python
def somme(n):
    ''' Renvoie la somme des entiers de 1 à n inclus'''
    s = 0
    for i in range(1, n+1):
        s = s + i
    return s
```

Prototypage de la boucle WHILE en python

`condition1` est un booléen qui est évalué à chaque tour de boucle. Tant que `condition1` `True`, on effectue les instructions de la boucle. Lorsque `condition1` devient `False`, on sort de la boucle.

Code Python
```python
n = 0
u = 1
while u < 1000:
    n = n + 1
    u = u * 2
print(n)
print(u)
```

Code Python
```python
mot = 'abracadabra'
c = 0
trouve = False
while c < len(mot) and not(trouve):
    trouve = mot[c] == 'r'
    c = c + 1
print(trouve)
```

AP05-S LISTES ET TUPLES

Définition 1 : Les listes

Indice i	0	1	2	3	4	5	6	7
Elément	T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]

Accéder aux éléments d'une liste

Code Python

```python
ma_liste = [1, 2, 3, 4, 5]
print(ma_liste[0])      # Affiche: 1
print(ma_liste[2])   # Affiche: 3
print(len(ma_liste))  # Affiche: 5
```

Modifier les éléments d'une liste

Code Python

```python
ma_liste = [1, 2, 3]
ma_liste[0] = 10
print(ma_liste)      # Affiche: [10, 2, 3]
print(len(ma_liste))  # Affiche: 3
```

Itérer sur une liste avec une boucle FOR

Code Python

```python
ma_liste = [4, 8, 7, 9, 4]
for i in range(0, len(ma_liste)):
    print(ma_liste[i])
```

Code Python

```python
ma_liste = [4, 8, 7, 9, 4]
for elt in ma_liste:
    print(elt)
```

Sommes des éléments d'une liste

Code Python

```python
def somme(liste):
    somme = 0
    for elt in liste:
        somme = somme + elt
    return somme
#
```

Recherche de maximum dans une liste

Code Python

```python
def maximum(liste):
    maximum = liste[0]
    for elt in liste:
        if elt > maximum:
            maximum = elt
    return maximum
```

Recherche présence d'un élément dans une liste

Code Python

```python
def est_present(elt, liste):
    for e in liste:
        if e == elt:
            return True
    return False
```

Vérification liste triée

Code Python

```python
def est_triee(liste):
    for i in range(0, len(liste)-1):
        if liste[i] > liste[i+1]:
            return False
    return True
```

Liste de listes

Code Python

```python
ma_liste = [[1, 2, 3], [4, 5], [7, 8, 9, 10]]
print(len(ma_liste))    # Affiche: 3
print(len(ma_liste[1])) # Affiche: 2
print(ma_liste[0])         # Affiche: [1, 2, 3]
print(ma_liste[1])        # Affiche: [4, 5]
print(ma_liste[2])        # Affiche: [7, 8, 9, 10]
print(ma_liste[0][1])     # Affiche: 2
print(ma_liste[1][1])     # Affiche: 5
print(ma_liste[2][0])     # Affiche: 7
```

Sommes des éléments d'une liste de listes

Code Python

```python
def somme(matrice):
    somme = 0
    for liste in matrice:
        for elt in liste:
            somme = somme + elt
    return somme
#
```

Recherche de maximum liste de listes

Code Python

```python
def maximum(matrice):
    maximum = matrice[0][0]
    for i in range(0, len(matrice)):
        for j in range(0, len(matrice[i])):
            if matrice[i][j] > maximum:
                maximum = matrice[i][j]
    return maximum
```

Quelques méthodes sur les listes

- `append(elt)` : ajoute l'élément `elt` à la fin de la liste
- `pop()` : supprime et renvoie le dernier élément de la liste
- `pop(i)` : supprime et renvoie l'élément d'indice `i` de la liste

Code Python

```python
ma_liste = [1, 2, 3, 4, 5]
ma_liste.append(6)
a = ma_liste.pop()
b = ma_liste.pop(2)
```

Créer une liste par itération

Code Python

```python
ma_liste = []
for i in range(0, 5):
    ma_liste.append(i)

print(ma_liste)
# Affiche: [0, 1, 2, 3, 4]
```

Code Python

```python
ma_liste = []
for i in range(0, 5):
    ma_liste = [i] + ma_liste

print(ma_liste)
# Affiche: [4, 3, 2, 1, 0]
```

Créer une liste par compréhension

Code Python

```python
ma_liste = [i for i in range(0, 5)]
print(ma_liste)
# Affiche: [0, 1, 2, 3, 4]
```

Le hasard et les listes

- La fonction `random.choice(liste)` permet de choisir un élément au hasard dans une liste.
- La fonction `random.shuffle(liste)` permet de mélanger les éléments d'une liste de manière aléatoire.

Le slicing

Le slicing permet de créer une nouvelle liste à partir d'une liste existante en sélectionnant une portion de cette liste.

Code Python

```python
ma_liste = [1, 2, 3, 4, 5]
print(ma_liste[1:4])    # Affiche: [2, 3, 4]
print(ma_liste[:3])        # Affiche: [1, 2, 3]
print(ma_liste[2:4])    # Affiche: [3, 4]
```

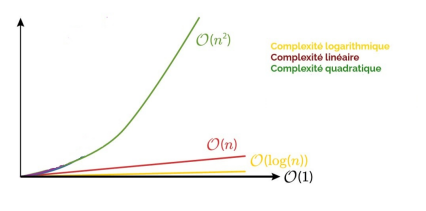
Définition 2 : Les tuples

Un **tuple** est une structure de donnée qui contient des éléments ou items identifiés par des indices. Les tuples ont les mêmes caractéristiques que les listes, à la différence que les éléments d'un tuple ne peuvent pas être modifiés après la création du tuple. Un tuple est une structure de donnée immuable.

AP06-S ALGORITHMES DE RECHERCHE

Définition 1 : Notion de complexité d'un algorithme

La complexité s'exprime en utilisant la notation **O** (grand O) et se déterminer en fonction de la taille de l'entrée du problème. On se place toujours dans le **pire des cas**



Définition 2 : Notion de terminaison d'un algorithme

Un algorithme est dit **terminant** s'il s'arrête après un nombre fini d'étapes pour toute entrée donnée.
La condition d'arrêt est appelée **variant de boucle**.

Définition 3 : Le variant de boucle

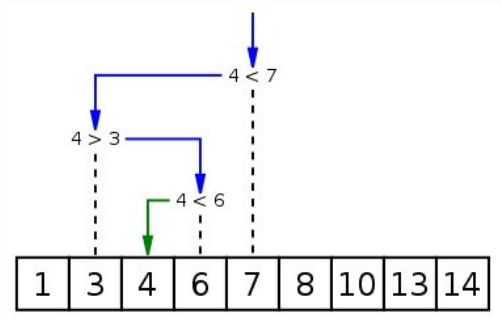
Un **variant de boucle** est une quantité qui est modifiée à chaque itération d'une boucle et qui permet de garantir la terminaison de l'algorithme. Pour qu'un variant de boucle soit efficace, il doit être :

- **positif ou nul** pour que la boucle se poursuive ;
- **décroissant** à chaque itération pour que la boucle se rapproche de sa condition d'arrêt.

Définition 4 : Notion de correction d'un algorithme

Un algorithme est dit **correct** s'il renvoie la bonne solution pour toute entrée donnée.
Un **invariant de boucle** est une propriété qui est vérifiée avant et après chaque itération d'une boucle. Si l'invariant de boucle est vérifié, alors l'algorithme est correct.

Définition 5 : Algorithme dichotomique



Code Python

```python
def recherche_dichotomique(val, L):
i_deb = 0
    i_fin = len(L) - 1
    while i_deb <= i_fin:
        i_cen = (i_deb + i_fin) // 2
        if L[i_cen] == val:
            return True
        elif L[i_cen] < val:
            i_deb = i_cen + 1
        else:
            i_fin = i_cen - 1
    return False
```

Définition 6 : Complexité logarithmique

Dans l'exemple précédent, on dira que cet algorithme possède une complexité **logarithmique** notée $O(\log(n))$

Propriété 1 : Détermination de complexité

- Si un algorithme n'effectue pas de boucle, alors sa complexité est de l'ordre de $O(1)$.
- Si un algorithme effectue une boucle FOR, alors sa complexité est de l'ordre de $O(n)$.
- Si un algorithme effectue k boucles FOR imbriquées, alors sa complexité est de l'ordre de $O(n^k)$.
- Si la taille du problème est divisée par 2, complexité est l'ordre de $O(\log(n))$ (WHILE).